

✖ Importações e instalações

```
1 !pip -q install osmnx networkx geopandas matplotlib pandas numpy scipy
```

```
1 import osmnx as ox
2 import networkx as nx
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import numpy as np
6 import time
7 import os
8 from IPython.display import display, HTML
```

✖ Parâmetros Experimentais

```
1 # Configura OSMnx
2 ox.settings.log_console = True
3 ox.settings.use_cache = True
4 ox.settings.timeout = 300
5
6 # Configura matplotlib para o Colab
7 plt.rcParams['figure.figsize'] = [12, 8]
8 plt.rcParams['figure.dpi'] = 100
```

✖ Pipeline de A* + MST

```
1 class CityPOIAnalyzer:
2     def __init__(self):
3         self.results = []
4         # Cria diretório no Colab
5         os.makedirs('/content/results/graphs', exist_ok=True)
6
7     def get_police_stations(self, place):
8         # Obtém apenas delegacias de polícia
9         print(f"Buscando delegacias em {place.split(',')[0]}...")
10
11         try:
12             tags = {'amenity': 'police'}
13             pois = ox.features.features_from_place(place, tags=tags)
14             police_points = []
15
16             for idx, row in pois.iterrows():
17                 if row.geometry.geom_type == 'Point':
18                     police_points.append((row.geometry.y, row.geometry.x))
19                 else:
20                     police_points.append((row.geometry.centroid.y, row.geometry.centroid.x))
21
22             print(f"{len(police_points)} delegacias encontradas")
23
24             # Mostra algumas delegacias encontradas
25             if len(police_points) > 0:
26                 print(f"Primeiras coordenadas: {police_points[:3]}")
27
28             return police_points
29
30         except Exception as e:
31             print(f"Erro ao buscar delegacias: {e}")
32             return []
33
34     def find_nearest_nodes(self, G, poi_points):
35         # Encontra os nós mais próximos para cada delegacia
36         print(f"Encontrando nós para {len(poi_points)} delegacias...")
37
38         # nearest_nodes espera: (graph, X, Y) onde X=longitude, Y=latitude
39         latitudes = [p[0] for p in poi_points] # Y = latitude
40         longitudes = [p[1] for p in poi_points] # X = longitude
41
42         nearest_nodes = ox.distance.nearest_nodes(G, longitudes, latitudes)
43         unique_nodes = list(set(nearest_nodes))
44
45         print(f"{len(unique_nodes)} nós únicos encontrados")
46
```

```

47     # Debug: mostrar se há nós duplicados
48     if len(unique_nodes) < len(nearest_nodes):
49         print(f" {len(nearest_nodes) - len(unique_nodes)} nós duplicados removidos")
50
51     return unique_nodes
52
53 def process_city(self, city_name):
54     # Processa uma cidade: baixa grafo, encontra delegacias, calcula rotas A* e MST
55     print(f"\n{' '*60}")
56     print(f" Processando {city_name}")
57     print(f"{' '*60}")
58
59     try:
60         # Baixa e preparar o grafo viário
61         print("1. Baixando grafo viário...")
62         G = ox.graph_from_place(city_name, network_type='drive')
63         print(f" Grafo baixado: {len(G.nodes())} nós, {len(G.edges())} arestas")
64
65         # Obtém delegacias
66         print("2. Buscando delegacias...")
67         police_points = self.get_police_stations(city_name)
68
69         if len(police_points) < 2:
70             print(f" Delegacias insuficientes ({len(police_points)}) em {city_name}. Pulando...")
71             return None
72
73         # Encontra nós mais próximos
74         print("3. Encontrando nós mais próximos...")
75         police_nodes = self.find_nearest_nodes(G, police_points)
76
77         if len(police_nodes) < 2:
78             print(f" Nós insuficientes ({len(police_nodes)}) para calcular MST. Pulando...")
79             return None
80
81         # Projeta o grafo para métrica UTM
82         print("4. Projetando grafo para UTM...")
83         G_proj = ox.projection.project_graph(G)
84
85         # Calcula grafo completo com A* entre delegacias
86         print("5. Calculando rotas A* entre delegacias...")
87         complete_graph = nx.Graph()
88         routes_calculated = 0
89         failed_routes = 0
90
91         total_pairs = len(police_nodes) * (len(police_nodes) - 1) // 2
92         print(f" Total de pares a calcular: {total_pairs}")
93
94         # Barra de progresso simples
95         from tqdm import tqdm
96         total_combinations = sum(1 for i in range(len(police_nodes)) for j in range(i+1, len(police_nodes)))
97
98         pbar = tqdm(total=total_combinations, desc=f"Rotas {city_name.split(',')[0]}")
99
100        for i in range(len(police_nodes)):
101            complete_graph.add_node(police_nodes[i])
102            for j in range(i + 1, len(police_nodes)):
103                try:
104                    # Calcula rota A* com heurística euclidiana
105                    path = nx.astar_path(
106                        G_proj,
107                        police_nodes[i],
108                        police_nodes[j],
109                        heuristic=lambda u, v: ox.distance.euclidean(
110                            G_proj.nodes[u]['y'], G_proj.nodes[u]['x'],
111                            G_proj.nodes[v]['y'], G_proj.nodes[v]['x']
112                        ),
113                        weight='length'
114                    )
115
116                    # Calcula comprimento total da rota
117                    path_length = sum(
118                        G_proj[path[k]][path[k+1]][0]['length']
119                        for k in range(len(path)-1)
120                    )
121
122                    complete_graph.add_edge(
123                        police_nodes[i],
124                        police_nodes[j],
125                        weight=path_length,
126                        path=path
127                    )
128                    routes_calculated += 1
129

```

```

130         except Exception as e:
131             failed_routes += 1
132         finally:
133             pbar.update(1)
134
135     pbar.close()
136     print(f" Rotas calculadas: {routes_calculated} (falhas: {failed_routes})")
137
138     # Calcular MST
139     if complete_graph.number_of_edges() > 0:
140         mst_edges = list(nx.minimum_spanning_edges(complete_graph, data=True))
141         total_mst_length = sum(edge[2]['weight'] for edge in mst_edges)
142
143         # Coletar rotas da MST
144         mst_routes = []
145         for u, v, data in mst_edges:
146             if 'path' in data:
147                 mst_routes.append(data['path'])
148
149         print(f"6. MST calculada: {total_mst_length/1000:.2f} km, {len(mst_routes)} rotas")
150         print(f"{city_name.split(',')[0]} processada com sucesso!")
151
152         return {
153             'city': city_name,
154             'graph': G_proj,
155             'police_nodes': police_nodes,
156             'complete_graph': complete_graph,
157             'mst_edges': mst_edges,
158             'mst_routes': mst_routes,
159             'total_mst_length': total_mst_length,
160             'num_police_stations': len(police_points),
161             'num_unique_nodes': len(police_nodes),
162             'num_routes': len(mst_routes)
163         }
164     else:
165         print("Nenhuma rota válida calculada")
166         return None
167
168     except Exception as e:
169         print(f"Erro processando {city_name}: {e}")
170         import traceback
171         traceback.print_exc()
172         return None
173
174     def analyze_all_cities(self, cities):
175         # Analisa todas as cidades
176         successful_cities = 0
177
178         print("INICIANDO ANÁLISE COMPLETA")
179         print("=" * 50)
180
181         for i, city in enumerate(cities, 1):
182             print(f"\n [{i}/{len(cities)}] Próxima cidade: {city}")
183             result = self.process_city(city)
184             if result:
185                 self.results.append(result)
186                 successful_cities += 1
187
188         print(f"\n{'='*60}")
189         print(f"RESUMO: {successful_cities} de {len(cities)} cidades processadas com sucesso")
190         print(f"{'='*60}")
191
192         return self.generate_report()
193
194     def generate_report(self):
195         # Gera relatório comparativo
196         if not self.results:
197             print(" Nenhum resultado válido para gerar relatório")
198             return None
199
200         report_data = []
201
202         for result in self.results:
203             num_edges_mst = len(result['mst_edges'])
204             avg_edge_length = result['total_mst_length'] / num_edges_mst if num_edges_mst > 0 else 0
205
206             report_data.append({
207                 'Cidade': result['city'].split(',')[0],
208                 'Total MST (km)': round(result['total_mst_length'] / 1000, 2),
209                 'Delegacias': result['num_police_stations'],
210                 'Nós Únicos': result['num_unique_nodes'],
211                 'Arestas na MST': num_edges_mst,

```

```

212         'Média por Aresta (km)': round(avg_edge_length / 1000, 2),
213         'Média por Delegacia (km)': round(result['total_mst_length'] / result['num_police_stations'] / 1000,
214     })
215
216     df = pd.DataFrame(report_data)
217
218     # Estilizar a tabela para melhor visualização
219     styled_df = df.style\
220         .background_gradient(subset=['Total MST (km)'], cmap='YlOrRd')\
221         .background_gradient(subset=['Delegacias'], cmap='Blues')\
222         .format({
223             'Total MST (km)': '{:.2f}',
224             'Média por Aresta (km)': '{:.2f}',
225             'Média por Delegacia (km)': '{:.2f}'
226         })
227
228     print("\n" + "="*80)
229     print(" RELATÓRIO COMPARATIVO - MST ENTRE DELEGACIAS")
230     print("="*80)
231     display(styled_df)
232
233     # Salvar resultados
234     df.to_csv('/content/results/comparative_analysis.csv', index=False)
235
236     if len(report_data) > 0:
237         total_km = sum(item['Total MST (km)'] for item in report_data)
238         avg_km = total_km / len(report_data)
239         max_city = max(report_data, key=lambda x: x['Total MST (km)'])
240         min_city = min(report_data, key=lambda x: x['Total MST (km)'])
241
242         print(f"\n ESTATÍSTICAS GERAIS:")
243         print(f"     Distância total MST: {total_km:.2f} km")
244         print(f"     Média por cidade: {avg_km:.2f} km")
245         print(f"     Maior MST: {max_city['Cidade']} ({max_city['Total MST (km)']} km)")
246         print(f"     Menor MST: {min_city['Cidade']} ({min_city['Total MST (km)']} km)")
247
248     return df
249
250     def plot_results(self):
251         # Plota resultados para cada cidade
252         if not self.results:
253             print(" Nenhum resultado para plotar")
254             return
255
256         print(f"\n Gerando gráficos para {len(self.results)} cidades...")
257
258         for result in self.results:
259             try:
260                 city_name_short = result['city'].split(',')[0]
261                 print(f" Plotando {city_name_short}...")
262
263                 # Criar figura maior para melhor visualização
264                 fig, ax = plt.subplots(figsize=(15, 12))
265
266                 # Plotar o grafo base
267                 ox.plot_graph(
268                     result['graph'],
269                     ax=ax,
270                     node_size=0,
271                     edge_color="lightgray",
272                     edge_linewidth=0.3,
273                     show=False,
274                     close=False
275                 )
276
277                 # Plotar rotas da MST em vermelho
278                 for route in result['mst_routes']:
279                     x = [result['graph'].nodes[n]['x'] for n in route]
280                     y = [result['graph'].nodes[n]['y'] for n in route]
281                     ax.plot(x, y, color='red', linewidth=3, zorder=4, alpha=0.8)
282
283                 # Plotar delegacias em azul
284                 police_x = [result['graph'].nodes[n]['x'] for n in result['police_nodes']]
285                 police_y = [result['graph'].nodes[n]['y'] for n in result['police_nodes']]
286                 ax.scatter(police_x, police_y, c='blue', s=100, zorder=5,
287                     edgecolor='black', linewidth=1.5, label='Delegacias')
288
289                 plt.title(f" MST entre Delegacias - {city_name_short}\n"
290                     f" Total: {result['total_mst_length']/1000:.2f} km | "
291                     f" Delegacias: {result['num_police_stations']}",
292                     fontsize=14, pad=20)
293                 plt.legend(fontsize=12)
294

```

```
295         plt.tight_layout()
296         filename = f'/content/results/graphs/mst_{city_name_short.lower().replace(" ", "_")}.png'
297         plt.savefig(filename, dpi=150, bbox_inches='tight')
298         plt.close()
299
300         print(f" Gráfico salvo: {filename}")
301
302     except Exception as e:
303         print(f" Erro ao plotar {result['city']}: {e}")
304
305 # Cidades para análise
306 CITIES = [
307     "Maceió, Alagoas, Brazil",
308     "Salvador, Bahia, Brazil",
309     "Fortaleza, Ceará, Brazil",
310     "São Luís, Maranhão, Brazil",
311     "João Pessoa, Paraíba, Brazil",
312     "Recife, Pernambuco, Brazil",
313     "Teresina, Piauí, Brazil",
314     "Natal, Rio Grande do Norte, Brazil",
315     "Aracaju, Sergipe, Brazil"
316 ]
317
318 # Executar análise
319 if __name__ == "__main__":
320     print(" Configurando ambiente...")
321
322     # Instalar tqdm para barra de progresso
323     try:
324         from tqdm import tqdm
325     except:
326         !pip install tqdm
327         from tqdm import tqdm
328
329     analyzer = CityPOIANalyzer()
330
331     print(" INICIANDO ANÁLISE DE MST ENTRE DELEGACIAS")
332     print(" Cidades a serem analisadas:")
333     for i, city in enumerate(CITIES, 1):
334         print(f" {i}. {city}")
335
336     print("\n Buscando apenas delegacias (amenity: police)")
337     print("=" * 50)
338
339     start_time = time.time()
340     df = analyzer.analyze_all_cities(CITIES)
341     analyzer.plot_results()
342     end_time = time.time()
343
344     print(f"\n Análise concluída em {end_time - start_time:.2f} segundos!")
345     print(" Relatório salvo em: /content/results/comparative_analysis.csv")
346     print(" Gráficos salvos em: /content/results/graphs/")
347
348     # Mostrar links para download no Colab
349     print("\n Para baixar os resultados:")
350     print(" from google.colab import files")
351     print(" files.download('/content/results/comparative_analysis.csv')")
352     print(" !zip -r /content/results.zip /content/results/")
353     print(" files.download('/content/results.zip')")
```



```
Configurando ambiente...
INICIANDO ANÁLISE DE MST ENTRE DELEGACIAS
Cidades a serem analisadas:
1. Maceió, Alagoas, Brazil
2. Salvador, Bahia, Brazil
3. Fortaleza, Ceará, Brazil
4. São Luís, Maranhão, Brazil
5. João Pessoa, Paraíba, Brazil
6. Recife, Pernambuco, Brazil
7. Teresina, Piauí, Brazil
8. Natal, Rio Grande do Norte, Brazil
9. Aracaju, Sergipe, Brazil

Buscando apenas delegacias (amenity: police)
=====
INICIANDO ANÁLISE COMPLETA
=====

[1/9] Próxima cidade: Maceió, Alagoas, Brazil

=====
Processando Maceió, Alagoas, Brazil
=====
1. Baixando grafo viário...
Grafo baixado: 16064 nós, 39603 arestas
2. Buscando delegacias...
Buscando delegacias em Maceió...
27 delegacias encontradas
Primeiras coordenadas: [(-9.6562285, -35.711526), (-9.6244851, -35.7388587), (-9.657386, -35.7559092)]
3. Encontrando nós mais próximos...
Encontrando nós para 27 delegacias...
27 nós únicos encontrados
4. Projetando grafo para UTM...
5. Calculando rotas A* entre delegacias...
Total de pares a calcular: 351
Rotas Maceió: 100%|██████████| 351/351 [00:09<00:00, 35.17it/s]
Rotas calculadas: 351 (falhas: 0)
6. MST calculada: 75.38 km, 26 rotas
Maceió processada com sucesso!

[2/9] Próxima cidade: Salvador, Bahia, Brazil

=====
Processando Salvador, Bahia, Brazil
=====
1. Baixando grafo viário...
Grafo baixado: 28213 nós, 64798 arestas
2. Buscando delegacias...
Buscando delegacias em Salvador...
83 delegacias encontradas
Primeiras coordenadas: [(-13.0073412, -38.505073), (-12.994457, -38.459707), (-12.926778, -38.4529509)]
3. Encontrando nós mais próximos...
Encontrando nós para 83 delegacias...
66 nós únicos encontrados
17 nós duplicados removidos
4. Projetando grafo para UTM...
5. Calculando rotas A* entre delegacias...
Total de pares a calcular: 2145
Rotas Salvador: 100%|██████████| 2145/2145 [01:18<00:00, 27.36it/s]
Rotas calculadas: 2087 (falhas: 58)
6. MST calculada: 139.08 km, 65 rotas
Salvador processada com sucesso!

[3/9] Próxima cidade: Fortaleza, Ceará, Brazil

=====
Processando Fortaleza, Ceará, Brazil
=====
1. Baixando grafo viário...
Grafo baixado: 36750 nós, 96222 arestas
2. Buscando delegacias...
Buscando delegacias em Fortaleza...
79 delegacias encontradas
Primeiras coordenadas: [(-3.7089995, -38.4691259), (-3.8142956, -38.5693326), (-3.8264887, -38.5588803)]
3. Encontrando nós mais próximos...
Encontrando nós para 79 delegacias...
74 nós únicos encontrados
5 nós duplicados removidos
4. Projetando grafo para UTM...
5. Calculando rotas A* entre delegacias...
Total de pares a calcular: 2701
Rotas Fortaleza: 100%|██████████| 2701/2701 [01:57<00:00, 23.02it/s]
Rotas calculadas: 2690 (falhas: 11)
6. MST calculada: 132.12 km, 73 rotas
Fortaleza processada com sucesso!

[4/9] Próxima cidade: São Luís, Maranhão, Brazil

=====
Processando São Luís, Maranhão, Brazil
```